TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 12: Producer, Consumer and the Other Tools

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Part 1: Producer and consumer **5**

1.1 Source code and execution of `prodcons.c` 5

1.2 Observations for Part 1 6

2 Part 2: The condition variable **8**

2.1 Source code and execution of `condvar.c` 8

2.2 Observations for Part 2 9

3 Part 3: Readers and writers **10**

3.1 Source code and execution of `readers.c` 10

3.2 Observations for Part 3 11

4 Part 4: Build a deadlock, then remove it **12**

4.1 Reproducing and fixing the deadlock (`deadlock.c`) 12

4.2 Observations for Part 4 13

Conclusions **15**

ABSTRACT

The goal of this laboratory is to combine multiple POSIX synchronization primitives—counting semaphores, mutexes, and condition variables—to solve classic concurrency problems (bounded-buffer producer-consumer and readers-writers) and to construct, analyze, and eliminate a multi-mutex deadlock. In Part 1, two counting semaphores (`empty` initialized to 4, `full` initialized to 0) and a mutex coordinate a fast producer and a slow consumer around a 4-slot circular buffer (`prodcons.c`), and swapping the mutex lock ahead of `sem_wait(&empty)` demonstrates how holding a mutex while blocking on a semaphore causes a deadlock. In Part 2, a mutex, a state variable (`ready`), and a condition variable (`changed`) solve the writer-before-reader ordering problem (`condvar.c`) using a `while (!ready)` loop. In Part 3, two semaphores (`room` and `rlock`) allow three readers to read simultaneously while keeping writers exclusive (`readers.c`). In Part 4, an AB–BA lock-ordering deadlock (`deadlock.c`) is observed with `timeout` (exit code 124), tested without `sleep(1)`, and eliminated by acquiring mutex `A` before mutex `B` in both threads. **Keywords:** Linux, pthreads, producer-consumer, bounded buffer, condition variable, readers-writers, starvation, deadlock, lock ordering

INTRODUCTION

Goal of the laboratory (Scopul lucrării)

Whereas Lab 11 used one synchronization primitive at a time, the purpose of this laboratory work is to make multiple synchronization tools work together on Linux to solve classic operating-system concurrency problems and to study deadlock creation and removal. Specifically, the scope and objectives of the laboratory are:

1. **Part 1 (Producer and consumer):** Build and run `prodcons.c`, where a fast producer inserts 8 items into a 4-slot circular buffer and a slow consumer removes them using two counting semaphores (`empty = 4`, `full = 0`) and a mutex (`lock`), and explain why locking `lock` before `sem_wait(&empty)` deadlocks when the buffer fills up.
2. **Part 2 (The condition variable):** Build and run `condvar.c`, which solves the `order.c` problem using a boolean state variable (`ready`), a mutex (`lock`), and a condition variable (`changed`), and explain why `pthread_cond_wait` must be guarded by `while (!ready)` rather than `if (!ready)`.
3. **Part 3 (Readers and writers):** Build and run `readers.c`, where two semaphores (`room` and `rlock`) and a shared counter (`readers`) allow multiple readers into the room together while writers enter alone, and explain the role of `rlock` and how continuous reader arrivals cause writer starvation.
4. **Part 4 (Build a deadlock, then remove it):** Build and run `deadlock.c`, where two threads lock mutexes `A` and `B` in opposite order and hang (`exit code: 124` under `timeout 5`), test the original code with `sleep(1)` removed across 10 runs, and fix the deadlock permanently by locking `A` before `B` in both threads.

All programs were compiled on Linux (openSUSE Tumbleweed, x86_64) using `gcc -Wall -pthread`. Complete command transcripts are saved in `evidence/` and `report.md`.

1 PART 1: PRODUCER AND CONSUMER

1.1 Source code and execution of `prodcons.c`

In `prodcons.c`, a 4-slot circular buffer (`buf[SIZE]`) is shared between a fast producer and a slow consumer (`usleep(300000)` per item). Three primitives coordinate access: `empty` counts free slots (starts at 4), `full` counts filled slots (starts at 0), and `lock` protects `buf`, `in`, `out`, and `count`:

```
/* prodcons.c - producer and consumer around a buffer of 4 slots.
 *   empty = free slots (starts at SIZE), full = filled slots (starts at 0), lock = the buffer
    itself.
 * Build: gcc -Wall -pthread -o prodcons prodcons.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE  4
#define ITEMS 8

int buf[SIZE];
int in = 0, out = 0, count = 0;       /* count only for printing */

sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);             /* wait for a free slot */
        pthread_mutex_lock(&lock);
        buf[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("put  %d  (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&full);              /* one more filled slot */
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);              /* wait for something to take */
        pthread_mutex_lock(&lock);
        int item = buf[out];
        out = (out + 1) % SIZE;
        count--;
        printf("          take %d  (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&empty);             /* one more free slot */
        usleep(300000);               /* the consumer is slow */
    }
    return NULL;
}
```

```
int main(void) {
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&c, NULL, consumer, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Compiling and running `./prodcons` produces:

```
$ gcc -Wall -pthread -o prodcons prodcons.c
$ ./prodcons
put  1  (in buffer: 1)
put  2  (in buffer: 2)
put  3  (in buffer: 3)
put  4  (in buffer: 4)
          take 1  (in buffer: 3)
put  5  (in buffer: 4)
          take 2  (in buffer: 3)
put  6  (in buffer: 4)
          take 3  (in buffer: 3)
put  7  (in buffer: 4)
          take 4  (in buffer: 3)
put  8  (in buffer: 4)
          take 5  (in buffer: 3)
          take 6  (in buffer: 2)
          take 7  (in buffer: 1)
          take 8  (in buffer: 0)
```

1.2 Observations for Part 1

1. **The producer is fast, yet `"in buffer"` never goes above 4. Which call stops it?** The call `sem_wait(&empty);` at the beginning of each loop iteration in `producer()` stops it. Because `empty` is initialized to `SIZE` (4), once the producer has placed 4 items into the buffer without the consumer removing any, `empty` has been decremented 4 times to `0`. On the 5th item, `sem_wait(&empty)` blocks the producer until the consumer removes an item and calls `sem_post(&empty)`.
2. **Who waits at the start: the consumer or the producer? Which call makes it wait?** At the start, the **consumer** waits (it is created first via `pthread_create(&c, ...)` when the buffer is still empty). The call `sem_wait(&full);` makes it wait because `full` starts at `0` (`sem_init(&full, 0, 0)`), blocking the consumer until the producer inserts the first item and calls `sem_post(&full)`.
3. **In `producer()`, swap the first two lines: lock the mutex BEFORE `sem_wait(&empty)`. Run it. What happens when the buffer is full, and why?** Swapping `pthread_mutex_lock(&lock);` before `sem_wait(&empty);` in `producer()` and running under `timeout 2`:

   ```
   put  1  (in buffer: 1)
   put  2  (in buffer: 2)
   ```

```
put  3  (in buffer: 3)
          take 1  (in buffer: 2)
put  4  (in buffer: 3)
put  5  (in buffer: 4)
exit code: 124
```

As soon as `in buffer` reaches `4` (the buffer is full and `empty == 0`), **the program deadlocks (hangs forever)**:

- On the next iteration, `producer()` locks `lock` first and then calls `sem_wait(&empty)`. Because `empty` is `0`, the producer goes to sleep inside `sem_wait(&empty)` **while still holding `lock`**.
- When `consumer()` wakes from `usleep(300000)` to take the next item, it passes `sem_wait(&full)` and then calls `pthread_mutex_lock(&lock)`. Because the sleeping producer holds `lock`, the consumer blocks on `lock` and can never reach `sem_post(&empty)`.

Each thread waits for what the other holds: the producer holds `lock` and waits for `empty`, while the consumer holds the ability to post `empty` and waits for `lock`.

2 PART 2: THE CONDITION VARIABLE

2.1 Source code and execution of `condvar.c`

`condvar.c` solves the writer-before-reader ordering problem using an explicit integer condition `ready`, protected by `lock`, and a condition variable `changed`:

```
/* condvar.c - the order.c problem, solved with a mutex and a condition variable.
 * Build: gcc -Wall -pthread -o condvar condvar.c
 */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

char message[32];
int ready = 0;                          /* the condition itself */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t changed = PTHREAD_COND_INITIALIZER;

void *writer(void *arg) {
    sleep(1);
    pthread_mutex_lock(&lock);
    strcpy(message, "hello");
    ready = 1;
    printf("A: message written\n");
    pthread_cond_signal(&changed);    /* wake one waiter */
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *reader(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready) {                  /* while, not if: re-check after waking */
        printf("B: not ready, I sleep\n");
        pthread_cond_wait(&changed, &lock);   /* unlocks, sleeps, locks again */
    }
    printf("B: I read \"%s\"\n", message);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

Compiling and running `./condvar`:

```
$ gcc -Wall -pthread -o condvar condvar.c
$ ./condvar
B: not ready, I sleep
A: message written
B: I read "hello"
```

2.2 Observations for Part 2

1. **In `order.c` (Lab 11) the semaphore itself remembered that the message was ready. Here, which variable remembers it?** The normal integer variable **`ready`** (`int ready = 0;`, set to `1` by `writer()` while holding `lock`) remembers that the message has been written. Unlike a semaphore, a condition variable (`changed`) has no internal counter: if `pthread_cond_signal` is called before `reader()` sleeps, the signal is lost, but `reader()` sees `ready == 1` under `lock` and skips `pthread_cond_wait` altogether.
2. **Replace `"while (!ready)"` with `"if (!ready)"`, compile, run. Does it still work here?** Running the modified version with `if (!ready)` outputs:

   ```
   B: not ready, I sleep
   A: message written
   B: I read "hello"
   ```

   Yes, it still works in this small two-thread program because there is only one waiting reader thread, `writer()` sets `ready = 1` before calling `pthread_cond_signal(&changed)`, and no spurious wakeup occurred during the 1-second wait.
3. **Then why do we write `while`? Give one reason why a thread can wake up and still find `ready == 0`.** We write `while (!ready)` because a thread returning from `pthread_cond_wait` is not guaranteed that the condition holds:
   - *Spurious wakeups:* POSIX and the Linux kernel (`futex`) allow `pthread_cond_wait` to wake up occasionally even when no thread signaled the condition variable (e.g., when a signal is delivered or due to kernel wait-queue implementation details).
   - *Multiple waiters / intercepted state:* If two consumers/readers are waiting (or another thread locks `lock` before the woken thread re-acquires `lock`), the first thread to run may consume the work and reset `ready = 0` before the second woken thread acquires `lock`. Re-testing in a `while` loop ensures the thread goes back to sleep if `ready == 0`.

3 PART 3: READERS AND WRITERS

3.1 Source code and execution of `readers.c`

`readers.c` allows any number of readers into the room simultaneously while ensuring that a writer enters only when the room is completely empty:

```
/* readers.c - readers and writers with two semaphores.
 *   Many readers may read together; a writer must be alone.
 * Build: gcc -Wall -pthread -o readers readers.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define READERS 3
#define WRITERS 2

int value = 0;                          /* the shared data */
int readers = 0;                        /* how many are reading now */
sem_t rlock;                            /* protects the counter readers */
sem_t room;                             /* 1 = the room is free for a writer */

void *reader(void *arg) {
    long id = (long)arg;
    sem_wait(&rlock);
    if (++readers == 1)                 /* the first reader in locks out the writers */
        sem_wait(&room);
    sem_post(&rlock);

    printf("reader %ld reads %d  (readers inside: %d)\n", id, value, readers);
    usleep(200000);

    sem_wait(&rlock);
    if (--readers == 0)                 /* the last reader out lets the writers in */
        sem_post(&room);
    sem_post(&rlock);
    return NULL;
}

void *writer(void *arg) {
    long id = (long)arg;
    sem_wait(&room);                    /* alone in the room */
    value += 10;
    printf("writer %ld writes %d (alone)\n", id, value);
    usleep(200000);
    sem_post(&room);
    return NULL;
}

int main(void) {
    sem_init(&rlock, 0, 1);
    sem_init(&room, 0, 1);
    pthread_t r[READERS], w[WRITERS];
    pthread_create(&w[0], NULL, writer, (void *)1L);
    for (long i = 0; i < READERS; i++)
```

```
        pthread_create(&r[i], NULL, reader, (void *)(i + 1));
    pthread_create(&w[1], NULL, writer, (void *)2L);
    for (int i = 0; i < READERS; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < WRITERS; i++) pthread_join(w[i], NULL);
    return 0;
}
```

Compiling and running `./readers`:

```
$ gcc -Wall -pthread -o readers readers.c
$ ./readers
writer 1 writes 10 (alone)
reader 1 reads 10  (readers inside: 1)
reader 2 reads 10  (readers inside: 2)
reader 3 reads 10  (readers inside: 3)
writer 2 writes 20 (alone)
```

3.2 Observations for Part 3

1. **Three readers are inside at the same time, but a writer is always alone. Which two lines of `reader()` make that possible?** The two lines:

   ```
   if (++readers == 1)              /* the first reader in locks out the writers */
       sem_wait(&room);
   ```

   (together with the matching exit check `if (--readers == 0) sem_post(&room);`). Only the **first** arriving reader (`++readers == 1`) calls `sem_wait(&room)` to lock out writers; the 2nd and 3rd readers see `++readers == 2` and 3, skip `sem_wait(&room)`, and read concurrently while the first reader holds `room` locked until the last reader (`--readers == 0`) unlocks it.
2. **Why does the counter `readers` need its own lock (`rlock`)? What could go wrong without it?** Because multiple reader threads concurrently increment (`++readers`), decrement (`--readers`), and branch on `readers`, leaving it unprotected causes a data race:
   - Two readers arriving together could both read `readers == 0` and both evaluate `++readers == 1` (or lose an increment so `readers` is `1` when two readers are inside). If both call `sem_wait(&room)`, the second reader blocks instead of reading concurrently; if an increment is lost, the first reader to leave decrements `readers` to `0` and calls `sem_post(&room)`, allowing a writer into the room **while another reader is still reading**.
   - Similarly, a lost decrement on exit would prevent `readers` from ever reaching `0`, so `sem_post(&room)` would never be called and writers would block forever.
3. **Imagine readers that keep arriving without a pause. What happens to `writer 2`? (This problem has a name: starvation.)** If new readers keep arriving before existing readers finish, there is always at least one reader in the room, so `readers` never drops back to `0`. Because `sem_post(&room)` is only executed when `--readers == 0`, `room` remains locked indefinitely and `writer 2` waits forever in `sem_wait(&room)`—a classic **writer starvation** caused by reader preference.

4 PART 4: BUILD A DEADLOCK, THEN REMOVE IT

4.1 Reproducing and fixing the deadlock (`deadlock.c`)

In the original version of `deadlock.c`, thread `one` locks `A` then `B`, whereas thread `two` locks `B` then `A`. Running it under `timeout 5`:

```
$ gcc -Wall -pthread -o deadlock deadlock.c
$ timeout 5 ./deadlock; echo "exit code: $?"
one: I have A, I want B
two: I have B, I want A
exit code: 124
```

Exit code `124` means `timeout` had to kill `./deadlock` after `5` seconds because neither thread could ever proceed.

Fixing `two()` so that it locks `A` first and then `B` (the same order as `one()`) yields the submitted `deadlock.c`:

```c
/* deadlock.c - two threads, two mutexes, fixed by locking in the same order.
 * Build: gcc -Wall -pthread -o deadlock deadlock.c
 * Run:   timeout 5 ./deadlock; echo "exit code: $?"
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void *one(void *arg) {
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);           /* waits for two */
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

void *two(void *arg) {
    pthread_mutex_lock(&A);           /* the fix: lock A first here too */
    printf("two: I have B, I want A\n");
    sleep(1);
    pthread_mutex_lock(&B);           /* waits for one */
    printf("two: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, one, NULL);
    pthread_create(&t2, NULL, two, NULL);
```

```
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("done\n");
    return 0;
}
```

Compiling and running the fixed `deadlock.c`:

```
$ gcc -Wall -pthread -o deadlock deadlock.c
$ timeout 5 ./deadlock; echo "exit code: $?"
one: I have A, I want B
one: I have both
two: I have B, I want A
two: I have both
done
exit code: 0
```

4.2 Observations for Part 4

1. **In the stuck run, which thread holds which mutex, and which one is each thread waiting for? Draw the circle.**
   - Thread `one` **holds mutex** `A` and is **waiting for mutex** `B` (`pthread_mutex_lock(&B)`).
   - Thread `two` **holds mutex** `B` and is **waiting for mutex** `A` (`pthread_mutex_lock(&A)`).

   The circular wait graph is:

   ```
        holds                    wants
   [A] -------> (Thread one) --------> [B]
    ^                                   |
    |   wants                   holds   v
    +---------- (Thread two) <----------+
   ```

2. **Remove both `sleep(1)` lines from the ORIGINAL version and run it ten times. Does it still always hang? Why is that worse, not better?** Running the original opposite-order program without `sleep(1)` ten times produced **9 hangs (`exit code: 124`) and 1 completion (`exit code: 0` on run 10)**:

   ```
   === run 9 ===
   one: I have A, I want B
   two: I have B, I want A
   exit code: 124
   === run 10 ===
   one: I have A, I want B
   one: I have both
   two: I have B, I want A
   two: I have both
   done
   exit code: 0
   ```

   No, it does not always hang: without `sleep(1)`, thread `one` sometimes finishes acquiring `A`, acquiring `B`, and unlocking both before the OS has even finished starting thread `two`. **Why this is worse, not better:** the deadlock bug is still present in the code, but now it is timing-dependent (a "Heisenbug") that can slip past testing and then freeze unpredictably in production whenever both

threads happen to overlap.

3. **In one sentence: why can taking the locks in the same order never deadlock?** Acquiring locks in a single global order (`A` before `B`) makes a circular wait impossible: any thread waiting for `B` already holds `A`, so whichever thread holds `B` cannot be waiting for `A` and is guaranteed to finish and release `B`.

CONCLUSIONS

This laboratory combined semaphores, mutexes, and condition variables to solve standard synchronization problems and analyze deadlock:

- In **Part 1**, two counting semaphores (`empty`, `full`) and a mutex coordinated a bounded-buffer producer and consumer (`prodcons.c`), and swapping the mutex lock before `sem_wait(&empty)` proved that a thread must never sleep on a semaphore while holding a mutex needed by the waking thread.
- In **Part 2**, `condvar.c` separated the boolean state (`ready`, protected by `lock`) from the sleep queue (`changed`), demonstrating why `pthread_cond_wait` always requires a `while (!ready)` loop.
- In **Part 3**, `readers.c` used `rlock` and `room` so the first reader locks out writers and the last reader lets them back in, allowing concurrent readers while illustrating reader-preference writer starvation.
- In **Part 4**, `deadlock.c` demonstrated how acquiring two mutexes in opposite order creates a circular wait and how enforcing a strict global lock order (`A` then `B`) eliminates deadlock permanently.